SQL

Structured Query Language

LENGUAJE SQL

Lenguaje SQL (Structured Query Language)

- Es el estándar para interactuar con bases de datos relacionales.
- Permite definir la estructura de los datos, manipularlos y consultarlos de forma eficiente.
- Se organiza en varios sublenguajes

DDL (Data Definition Language):

- Se usa para definir y modificar la estructura de la base de datos, como crear o eliminar tablas, y establecer relaciones.
- Sentencias: CREATE TABLE, ALTER TABLE.

DML (Data Manipulation Language):

- Permite insertar, actualizar o eliminar registros en las tablas.
- Sentencias: INSERT, UPDATE, DELETE.

DQL (Data Query Language):

- Se utiliza para realizar consultas y obtener información.
- Sentencia principal SELECT.

DCL (Data Control Language):

- Gestiona permisos y control de acceso a los datos.
- Sentencias: GRANT, REVOKE.

TCL (Transaction Control Language):

- Controla transacciones, garantizando la coherencia de los datos.
- Sentencias: COMMIT, ROLLBACK.

Tema 4
DQL
(Data Query Language)

SELECT es la instrucción principal de SQL y permite realizar consultas a la base de datos.

- Es muy potente y flexible
- Consulta una o varias tablas
- Filtra filas y columnas
- Ordena resultados
- Agrupa datos (SUM, COUNT, etc.)
- Puede generar tablas nuevas a partir de consultas

Sentencia básica SELECT

Sintaxis

```
SELECT [DISTINCT]<cláusula>
[ INTO <cláusula>]
FROM <cláusula>
[ WHERE <cláusula>]
[ GROUP BY <cláusula>]
[ HAVING <cláusula>]
[ ORDER BY <cláusula>]
[ LIMIT num][OFFSET num]
```

Cláusulas obligatorias:

- **SELECT** → columnas que queremos mostrar
- **FROM** → tablas de donde proceden los datos

Cláusulas opcionales:

- **INTO** → permite crear una nueva tabla con el resultado de la consulta
- **WHERE** → condiciones o filtros
- **GROUP BY** → agrupaciones
- **HAVING** → filtros sobre grupos
- **ORDER BY** → ordenación
- **LIMIT / OFFSET** → limitar resultados

Ejecución de la sentencia SELECT

¿Sabes cual es el orden de ejecución de sus cláusulas?
No es lo que parece:

```
SELECT DISTINCT department_id
FROM employees
JOIN orders ON customers.customer_id =
orders.customer_id
WHERE salary > 3000
GROUP BY department
HAVING AVG(salary) > 5000
ORDER BY department
LIMIT 10 OFFSET 5;
```

- #1 • FROM employees
- #2 • JOIN orders ON customers.customer_id = orders.customer_id
- #3 • WHERE salary > 3000
- #4 • GROUP BY department
- #5 • HAVING AVG(salary) > 5000
- #6 y #7 • **SELECT DISTINCT department_id**
- #8 • ORDER BY department
- #9 • LIMIT 10 OFFSET 5;

Tipos de Consultas SELECT

Consulta Básica

- Sentencia básica con única tabla en el FROM

Consultas Avanzadas

- Combinación de tablas:
  - PRODUCTO CARTESIANO
  - INNER JOIN
  - NATURAL JOIN
  - OUTER JOIN
- Subconsultas:
  - Sintaxis en el FROM
  - Sintaxis en el WHERE o HAVING
  - Sintaxis en el SELECT
- Operaciones de Conjuntos:
  - UNION
  - INTERSECT
  - EXCEPT (DIFERENCIA)

Consultas Básicas

Cláusula SELECT

¿Para qué sirve?

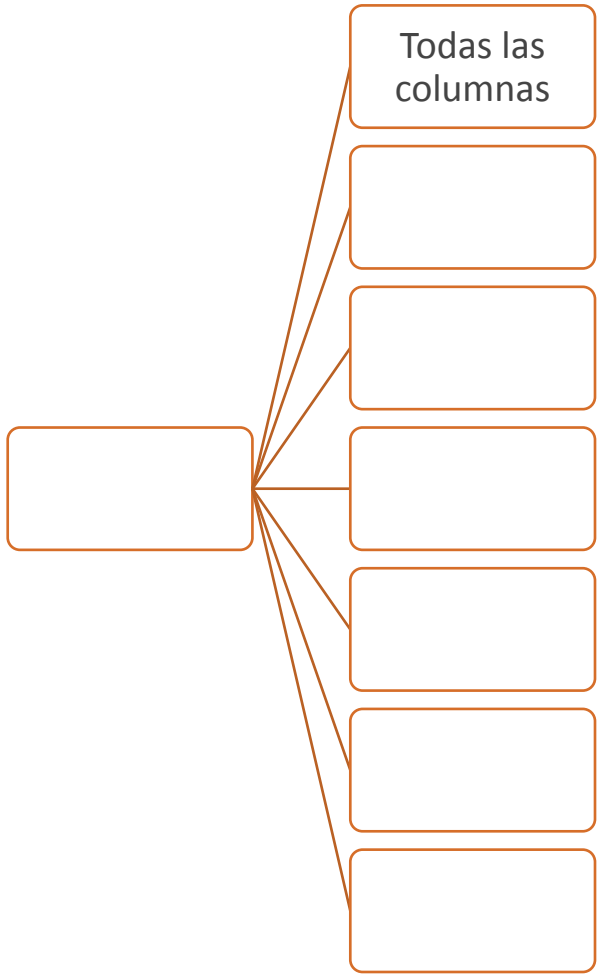
- Define qué datos se quieren ver en la consulta

¿Cuándo se utiliza?

- Seleccionar **columnas** concretas (por nombre) de una o varias tablas.
- Seleccionar **todas las columnas** usando el asterisco (*).
- Emplear **expresiones y cálculos**
- Utilizar **funciones de agregación**
- Asignar **alias** mediante **AS** a las columnas o resultados de expresiones
- Eliminar duplicados mediante **DISTINCT**
- Usar **subconsultas** como campos, para obtener resultados de otras consultas

Ejemplo SELECT

```
SELECT <cláusula>
[INTO] <cláusula>
FROM <cláusula>
[ WHERE <cláusula>]
[ GROUP BY <cláusula>]
[ HAVING <cláusula>]
[ ORDER BY <cláusula>]
[ LIMIT num] [OFFSET num2]
```



Ejemplos:

```
SELECT * FROM POBLACIONS;
```

```
SELECT nom, altura FROM POBLACIONS;
```

```
SELECT nom, poblacio, poblacio * 1.05 FROM POBLACIONS;
```

```
SELECT nom, latitud, SUBSTR(latitud,1,STRPOS(latitud,'º')-1)
FROM POBLACIONS;
```

```
SELECT cod_m AS "Codi Municipi" , nom FROM
POBLACIONS;
```

```
SELECT DISTINCT provincia FROM COMARQUES
```

```
SELECT nom, altura, altura - (SELECT AVG(altura) FROM
POBLACIONS) FROM POBLACIONS
```

SELECT
Alias (AS) del SELECT

```
SELECT <cláusula>
[INTO] <cláusula>
FROM <cláusula>
[ WHERE <cláusula>]
[ GROUP BY <cláusula>]
[ HAVING <cláusula>]
[ ORDER BY <cláusula>]
[ LIMIT n] [OFFSET m]
```

¿Qué es un alias?

- Un alias es un nombre alternativo para una columna en los resultados de una consulta.
- Se usa para mejorar la claridad y presentación.

¿Cuándo se utiliza?

- Para renombrar columnas a nombres más legibles.
- Para poner nombres a expresiones o campos calculados.
- Para mostrar cabeceras compuestas por varias palabras (con comillas dobles).

Ejemplos:

```
SELECT poblacio AS habitants
FROM POBLACIONS;
```

```
SELECT poblacio * 1.05 AS calculo FROM POBLACIONS;
```

```
SELECT nom_c || ' (' || provincia || ')' AS
"Comarca (provincia)"
FROM COMARQUES;
```

SELECT
Funciones de agregado

```
SELECT <cláusula>
[INTO] <cláusula>
FROM <cláusula>
[ WHERE <cláusula>]
[ GROUP BY <cláusula>]
[ HAVING <cláusula>]
[ ORDER BY <cláusula>]
[ LIMIT n] [OFFSET m]
```

¿Qué son?

- Son funciones que obtienen **un único resultado** a partir de los valores de una columna en un conjunto de filas.
- Estas funciones actúan sobre un conjunto de filas, que por defecto es toda la tabla.

¿Cuándo se utiliza?

- **COUNT(* | columna)**: Cuenta filas, si se usa una columna. *No cuenta valores nulos*.
- **SUM(columna)**: Devuelve la suma de los valores. Ignora valores nulos.
- **AVG(columna)**: Media aritmética. Ignora valores nulos.
- **MAX(columna):** Valor máximo.
- **MIN(columna):** Valor mínimo.
- **VAR_SAMP(columna)**: Varianza de una muestra.
- **STDDEV(columna)**: Desviación típica de una muestra.

Ejemplos:

```
SELECT COUNT(*)
FROM POBLACIONS;
```

```
SELECT SUM(poblacio) FROM POBLACIONS;
```

```
SELECT AVG(poblacio) FROM POBLACIONS;
```

```
SELECT AVG(poblacio/extensio) FROM
POBLACIONS;
```

```
SELECT MAX(altura), MIN(altura) FROM
POBLACIONS;
```

SELECT
Predicados ALL y DISTINCT

```
SELECT <cláusula>
[INTO] <cláusula>
FROM <cláusula>
[ WHERE <cláusula>]
[ GROUP BY <cláusula>]
[ HAVING <cláusula>]
[ ORDER BY <cláusula>]
[ LIMIT n] [OFFSET m]
```

¿Qué son?

- Controlan cómo se manejan los valores devueltos de las columnas seleccionadas.

¿Cuándo se utiliza ALL?

- Indica que se deben devolver todas las filas, incluyendo duplicados.
- Es el comportamiento por defecto de SQL y normalmente no se escribe

¿Cuándo se utiliza DISTINCT?

- Elimina filas duplicadas del resultado.
- La consulta devuelve solo filas únicas.
- Se pueden utilizar dentro de funciones de agregados

Ejemplos:

```
SELECT ALL cod_m FROM INSTITUTS
```

```
SELECT DISTINCT cod_m FROM INSTITUTS
```

```
SELECT COUNT(DISTINCT cod_m)
FROM INSTITUTS
```

```
SELECT DISTINCT codpostal FROM INSTITUTS
WHERE cod_m=12040
```

```
SELECT DISTINCT nom_c , llengua FROM
POBLACIONS ORDER BY 1
```

Cláusula FROM

¿Para qué sirve?

- Indica el origen de los datos que se van a consultar, especificando la tabla o tablas de las que se extraerá la información para la consulta.

¿Cuándo se utiliza?

- Para consultas sobre **una única tabla**
- Para combinar datos de varias tablas utilizando **JOINs**, permitiendo hacer consultas complejas sobre datos relacionados.
- Para incluir **subconsultas**, usando los resultados de otra consulta como fuente temporal de datos.

Ejemplo FROM

```
SELECT <cláusula>
[INTO] <cláusula>
FROM <cláusula>
[ WHERE <cláusula>]
[ GROUP BY <cláusula>]
[ HAVING <cláusula>]
[ ORDER BY <cláusula>]
[ LIMIT n] [OFFSET m]
```

Ejemplos:

```
SELECT COMARQUES.nom_c, nom
FROM COMARQUES,POBLACIONS;
```

```
SELECT COMARQUES.nom_c, POBLACIONS.nom
FROM COMARQUES INNER JOIN POBLACIONS ON COMARQUES.nom_c = POBLACIONS.nom_c
```

```
SELECT POBLACIONS.nom,INSTITUTS.nom
FROM POBLACIONS NATURAL JOIN INSTITUTS;
```

```
SELECT AVG(quants)
FROM (SELECT COUNT(*) AS quants FROM POBLACIONS GROUP BY nom_c) AS S ;
```

Cláusula WHERE

¿Para qué sirve?

- Para filtrar los datos y devolver solo aquellas filas que cumplen una **CONDICIÓN específica**

¿Cómo se utiliza?

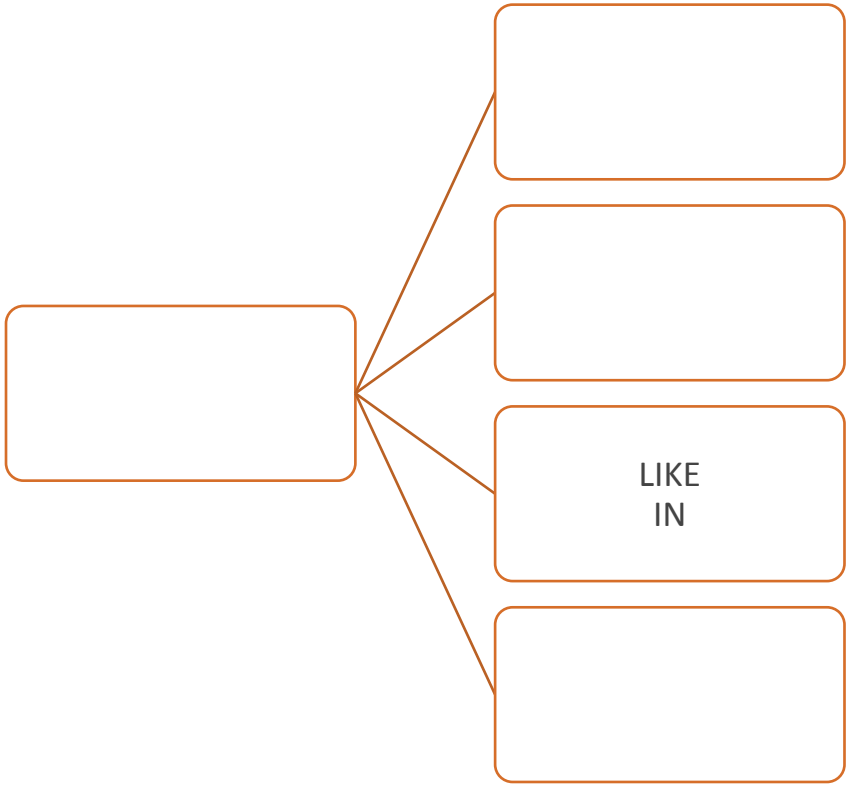
- **Operadores Lógicos:** **AND, OR, NOT**
  Para especificar más de una condición a la consulta
- **Operadores de Comparación:** **=, <, >, <=, >=, <>**
  Los posibles valores serán: *Campo de otra tabla, Números sin comillas , Texto y Fechas con comillas simples y valor nulo con* ***NULL***
- **Operadores especializados:**
  - Rangos: **BETWEEN .. AND**
  - Patrones: **LIKE** (con % y _)
  - Conjuntos: (NOT) **IN**
  - Valores nulos: **IS** (NOT) **NULL**

Importante

- No puede usarse directamente sobre campos agregados, para eso se usa HAVING.

Ejemplo WHERE

```
SELECT <cláusula>
[INTO] <cláusula>
FROM <cláusula>
[ WHERE <cláusula>]
[ GROUP BY <cláusula>]
[ HAVING <cláusula>]
[ ORDER BY <cláusula>]
[ LIMIT n] [OFFSET m]
```



Ejemplos:

```
SELECT * FROM COMARQUES WHERE provincia = 'Castelló';
```

```
SELECT * FROM POBLACIONS WHERE nom_c = 'Plana Alta' AND altura >
300;
```

```
SELECT * FROM INSTITUTS WHERE codpostal=12001 OR codpostal=12002
OR codpostal=12003;
```

```
SELECT * FROM INSTITUTS WHERE codpostal >= 12001 AND codpostal <=
12003;
```

```
SELECT * FROM INSTITUTS WHERE codpostal BETWEEN 12001 AND 12003;
```

```
SELECT * FROM INSTITUTS WHERE codpostal IN (12001,12002,12003);
```

```
SELECT nom, nom_c FROM POBLACIONS WHERE nom LIKE 'G%';
```

```
SELECT * FROM INSTITUTS WHERE numero IS NULL;
```

```
SELECT POBLACIONS.nom,INSTITUTS.nom FROM POBLACIONS , INSTITUTS
WHERE POBLACIONS.cod_m=INSTITUTS.cod_m;
```

```
SELECT * FROM POBLACIONS
WHERE altura > (SELECT AVG(altura) FROM POBLACIONS)
```

Cláusula GROUP BY

¿Para qué sirve?

- Se utiliza para agrupar filas que tienen valores idénticos en una o más columnas
- En vez de trabajar con filas individuales, esta cláusula permite obtener resultados resumidos para cada grupo.

¿Cuándo se utiliza?

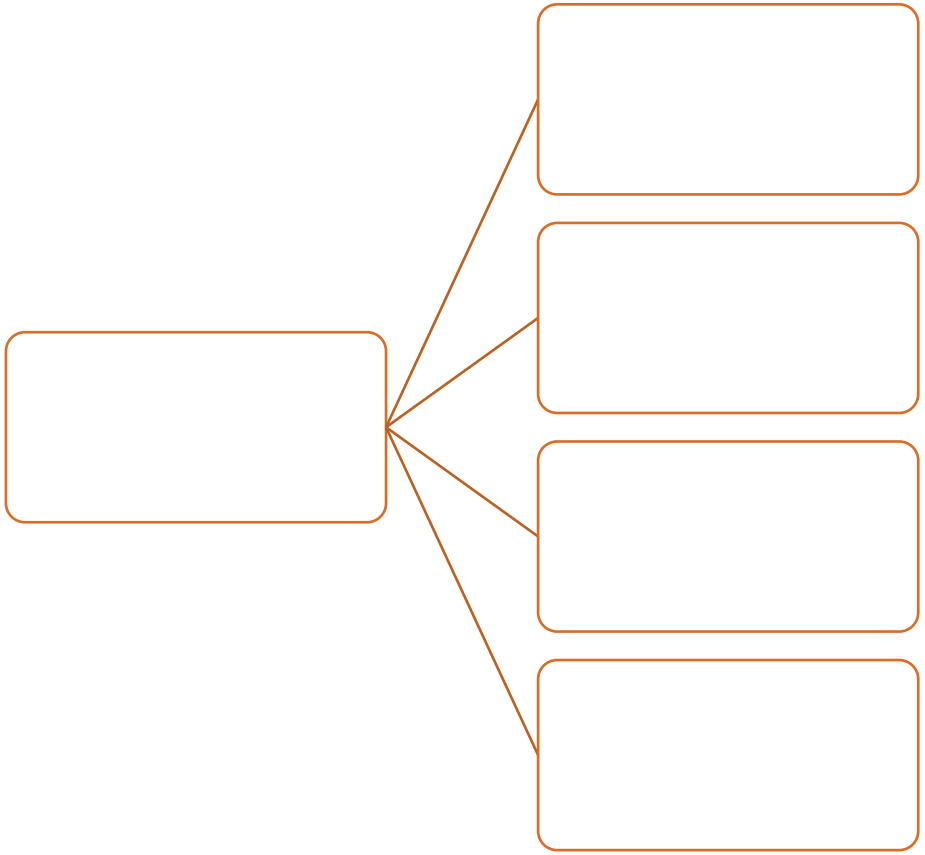
- Permite aplicar **funciones de agregación** (como SUM, COUNT, AVG) a cada grupo de datos resultante, de manera que devuelven un valor por grupo.
- **Combinada con HAVING**, permite filtrar los grupos resultantes en base al resultado de las funciones de agregación.

Importante:

- La cláusula WHERE se ejecuta antes que GROUP BY, por lo que la consulta se filtra antes de agrupar (ver orden de ejecución de cláusulas).
- Todas las columnas en SELECT deben estar en GROUP BY o dentro de una función de agregado.
- No permite Funciones de Agregación, ni posición de campos como ORDER BY

Ejemplo GROUP BY

```
SELECT <cláusula>
[INTO] <cláusula>
FROM <cláusula>
[ WHERE <cláusula>]
[ GROUP BY <cláusula>]
[ HAVING <cláusula>]
[ ORDER BY <cláusula>]
[ LIMIT n] [OFFSET m]
```



Ejemplos:

```
SELECT nom_c  FROM POBLACIONS
GROUP BY nom_c
```

```
SELECT COUNT(*) FROM POBLACIONS
WHERE cod_m >= 12000 and cod_m < 13000
GROUP BY nom_c
```

```
SELECT nom_c, (poblacio / extensio) as densidad
FROM POBLACIONS
GROUP BY nom_c, (poblacio / extensio)
```

```
SELECT cod_m, codpostal, COUNT(*) FROM INSTITUTS
GROUP BY cod_m, codpostal
```

```
SELECT cod_m, COUNT(*) FROM INSTITUTS
WHERE nom LIKE 'CIPFP%'
GROUP BY cod_m
HAVING COUNT(*) > 1
```

Cláusula HAVING BY

¿Para que sirve?

- Permite filtrar grupos, no filas individuales.
- Similar a cómo WHERE filtra las filas del SELECT, HAVING filtra grupos del GROUP BY.

¿Cuándo se utiliza?

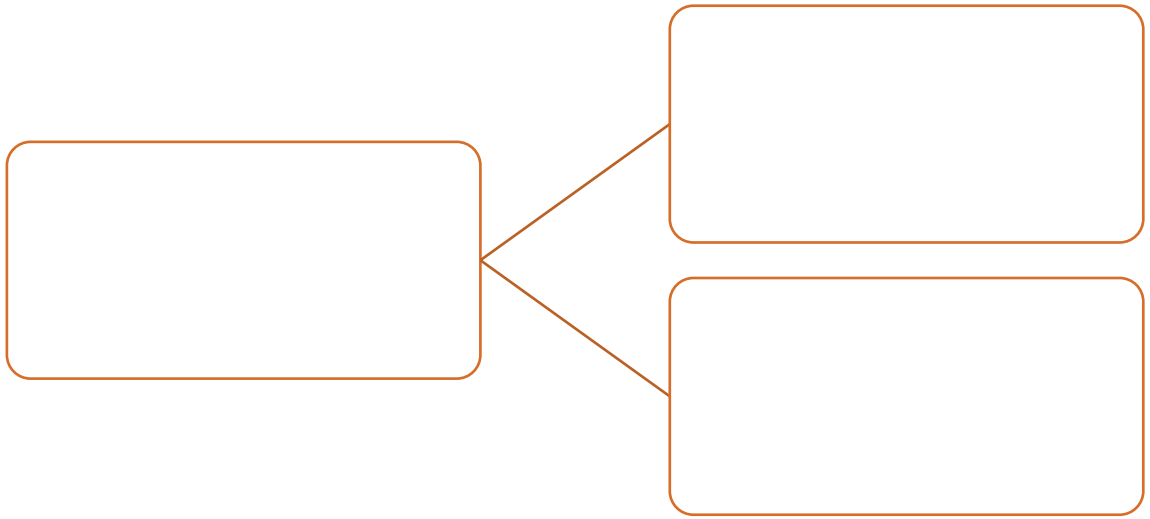
- Cuando la condición implica funciones de agregación (COUNT, AVG, SUM…).
- Cuando queremos seleccionar grupos ya formados por GROUP BY.

Importante:

- Cláusula que se usa junto a GROUP BY.

Ejemplo HAVING

```
SELECT <cláusula>
[INTO] <cláusula>
FROM <cláusula>
[ WHERE <cláusula>]
[ GROUP BY <cláusula>]
[ HAVING <cláusula>]
[ ORDER BY <cláusula>]
[ LIMIT n] [OFFSET m]
```



Ejemplos:

```
SELECT nom_c , AVG(altura) AS "Altura
mitjana" , SUM(poblacio) AS "Total
població" , Avg(poblacio) AS "Població
mitjana"
FROM POBLACIONS
GROUP BY nom_c
HAVING AVG(altura) > 800;
```

```
SELECT cod_m, COUNT(*) FROM INSTITUTS
WHERE nom LIKE 'CIPFP%'
GROUP BY cod_m
HAVING COUNT(*) > 1
```

Cláusula ORDER BY

¿Para qué sirve?

- Cláusula que permite ordenar los resultados de una consulta según el orden especificados, de manera ascendente (ASC) (por defecto) o descendente (DESC)

¿Cuándo se utiliza?

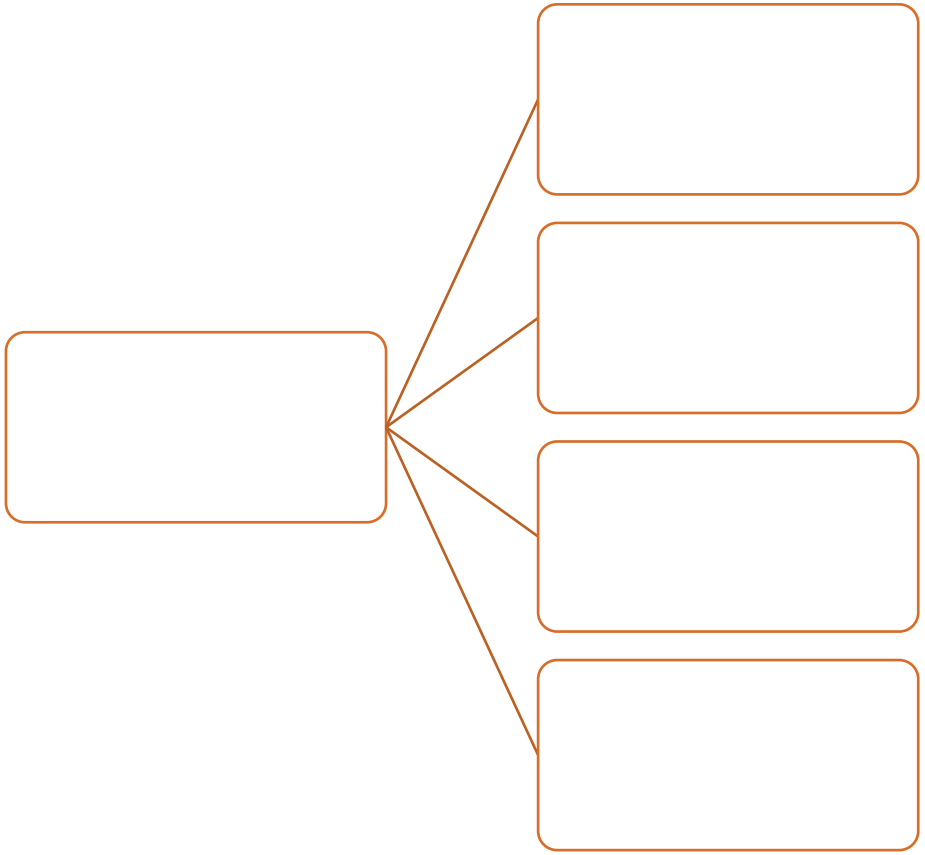
- Ordenar por **uno o varios campos**.
- Ordenar por **expresiones**, incluso si no aparecen en la lista de columnas mostradas.
- Ordenar por **posición** del campo en el SELECT

Importante:

- Si un campo no está indexado, ordenar puede ser más lento.
- Si se ordena frecuentemente por un mismo campo, conviene crear un índice (se verá más adelante).
- Si hay GROUP BY o funciones de agregado, solo se puede ordenar por campos del GROUP BY o agregados; de lo contrario, se produce error.

Ejemplo ORDER BY

```
SELECT <cláusula>
[INTO] <cláusula>
FROM <cláusula>
[ WHERE <cláusula>]
[ GROUP BY <cláusula>]
[ HAVING <cláusula>]
[ ORDER BY <cláusula>]
[ LIMIT n] [OFFSET m]
```



Ejemplos:

```
SELECT nom_c, provincia FROM COMARQUES
ORDER BY nom_c
```

```
SELECT nom_c, provincia FROM COMARQUES
ORDER BY provincia DESC, nom_c
```

```
SELECT nom, poblacio, extensio FROM POBLACIONS
ORDER BY poblacio/extensio DESC
```

```
SELECT nom, poblacio, extensio, poblacio/extensio FROM
POBLACIONS
ORDER BY 4 DESC
```

```
SELECT nom_c, COUNT(*), SUM(poblacio) FROM POBLACIONS
GROUP BY nom_c
HAVING COUNT(*) > 10
ORDER BY COUNT(*) , SUM(poblacio) DESC
```

Cláusulas LIMIT OFFSET

¿Para que sirve?

- Hace que no aparezcan todas las filas que devuelve una sentencia, sino solo algunas.

¿Cuándo se utiliza ?

- **LIMIT n : Para** especificar cuántas filas queremos mostrar.
- **OFFSET m : Para** indicar desde qué posición se empieza a mostrar (cuántas filas se saltan). Si no usamos OFFSET, se muestran las primeras filas.
- *n* y m deben ser un número entero.

Importante:

- Para que esta cláusula tenga sentido, es conveniente ordenar previamente los datos usando **ORDER BY**, ya que “las primeras” filas deben tener un orden definido.
- Los valores **NULL** aparecen antes en orden descendente. Por lo que se suelen omitir con esta cláusula.

Ejemplo LIMIT / OFFSET

```
SELECT <cláusula>
[INTO] <cláusula>
FROM <cláusula>
[ WHERE <cláusula>]
[ GROUP BY <cláusula>]
[ HAVING <cláusula>]
[ ORDER BY <cláusula>]
[ LIMIT n] [OFFSET m]
```

Ejemplos:

```
SELECT nom , altura FROM POBLACIONS
ORDER BY altura DESC
 LIMIT 10
```

```
SELECT nom , altura FROM POBLACIONS
ORDER BY altura DESC
 LIMIT 10 OFFSET 3
```

```
SELECT nom , altura FROM POBLACIONS WHERE altura IS NOT NULL
ORDER BY altura DESC
 LIMIT 10
```

```
SELECT nom , altura FROM POBLACIONS
ORDER BY altura DESC
 OFFSET 3
```

```
SELECT cod_m , COUNT(*) FROM INSTITUTS
GROUP BY cod_m
ORDER BY 2 DESC
LIMIT 10 OFFSET 3
```

Cláusula INTO

¿Para que sirve?

- Para **crear una tabla nueva**, con su estructura y sus datos (los que devuelve la sentencia SELECT).
- La nueva tabla incluye:
  - Datos devueltos por la consulta.
  - Mismos tipos de datos.
  - Nombres de columnas del SELECT (mejor usar alias).

¿Para que se utilizas?

- Crear copias de tablas.
- Generar tablas resumen con funciones de agregación (SUM, COUNT, AVG…).
- Trabajar temporalmente con datos limpios o transformados.

Importante:

- No se pueden definir claves primarias, foráneas ni restricciones.
- No forma parte del estándar ANSI SQL (característica específica de algunos motores).

Ejemplo INTO

```
SELECT <cláusula>
[INTO] <cláusula>
FROM <cláusula>
[ WHERE <cláusula>]
[ GROUP BY <cláusula>]
[ HAVING <cláusula>]
[ ORDER BY <cláusula>]
[ LIMIT n] [OFFSET m]
```

Ejemplos:

```
SELECT * INTO COMARQUES_COPIA
FROM COMARQUES
```

```
SELECT nom_c, COUNT(*) AS
num_pobles, SUM(poblacio) AS
poblacio , SUM(extensio) AS extensio
, AVG(altura) AS altura_mitjana
INTO RESUM_COMARQUES FROM POBLACIONS
GROUP BY nom_c
```

Consultas Avazadas